C > OS LAB > C ML.c > main()

```c
#include <stdio.h>
struct Process {
    int pid, at, bt, type;
    int ct, tat, wt;};
void sortByArrival(struct Process p[], int n){
    int i, j;
    struct Process temp;
    for(i = 0; i < n-1; i++)
    {
        for(j = i+1; j < n; j++)
        {
            if(p[i].at > p[j].at)
            {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}
int main()
{
    int n, i;
    struct Process p[20], sys[20], user[20];
    int sysCount = 0, userCount = 0;
    int time = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("Enter PID, AT, BT, TYPE (0=System,1=User): ");
        scanf("%d %d %d %d", &p[i].pid, &p[i].at, &p[i].bt, &p[i].type);
        if(p[i].type == 0)
            sys[sysCount++] = p[i];
        else
            user[userCount++] = p[i];}
    sortByArrival(sys, sysCount);
    sortByArrival(user, userCount);
```


```c
int main()
    sortByArrival(sys, sysCount);
    sortByArrival(user, userCount);
```


```c
    printf("\nGantt Chart:\n| ");
    for(i = 0; i < sysCount; i++){
        if(time < sys[i].at)
            time = sys[i].at;
        printf("P%d | ", sys[i].pid);
        time += sys[i].bt;
        sys[i].ct = time;
        sys[i].tat = sys[i].ct - sys[i].at;
        sys[i].wt = sys[i].tat - sys[i].bt;
    }
    for(i = 0; i < userCount; i++){
        if(time < user[i].at)
            time = user[i].at;
        printf("P%d | ", user[i].pid);
        time += user[i].bt;
        user[i].ct = time;
        user[i].tat = user[i].ct - user[i].at;
        user[i].wt = user[i].tat - user[i].bt;
    }
    printf("\n\n");
    printf("PID\tAT\tBT\tTYPE\tCT\tTAT\tWT\n");
    for(i = 0; i < sysCount; i++)
    {
        printf("P%d\t%d\t%d\tSystem\t%d\t%d\t%d\n",
        sys[i].pid, sys[i].at, sys[i].bt,
        sys[i].ct, sys[i].tat, sys[i].wt);
    }
    for(i = 0; i < userCount; i++){
        printf("P%d\t%d\t%d\tUser\t%d\t%d\t%d\n",
        user[i].pid, user[i].at, user[i].bt,
        user[i].ct, user[i].tat, user[i].wt);}
    return 0;
}
```

```
cd "/Users/sharan/Documents/Coding/C/OS LAB/" && gcc ML.c -o ML && "/Users/sharan/Documents/Coding/C/OS LAB/"ML
→  Coding cd "/Users/sharan/Documents/Coding/C/OS LAB/" && gcc ML.c -o ML && "/Users/sharan/Documents/Coding/C/OS LAB/"ML
Enter number of processes: 5
Enter PID, AT, BT, TYPE (0=System,1=User): 0 1 2 0
Enter PID, AT, BT, TYPE (0=System,1=User): 1 2 3 1
Enter PID, AT, BT, TYPE (0=System,1=User): 2 3 4 0
Enter PID, AT, BT, TYPE (0=System,1=User): 3 4 5 1
Enter PID, AT, BT, TYPE (0=System,1=User): 4 5 6 0

Gantt Chart:
| P0 | P2 | P4 | P1 | P3 |

PID     AT      BT      TYPE    CT      TAT     WT
P0      1       2       System  3       2       0
P2      3       4       System  7       4       0
P4      5       6       System  13      8       2
P1      2       3       User    16      14      11
P3      4       5       User    21      17      12
→  OS LAB git:(main) ✗
```